

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task 6.3

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Student Information System (Express.js)

2. AIM

To implement a RESTful web service using Express.js to manage student information.

3. OBJECTIVE

The objectives of this experiment are:

- Create an Express server.
 - Implement multiple routes (/, /students, /about).
 - Render HTML dynamically.
-

4. THEORY

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

5. ALGORITHM / PROCEDURE

1. Initialize the Express app.
 2. Define routes for home, students directory, and about page.
 3. Implement a 404 error handler.
 4. Start the server on a specified port.
-

6. PROGRAM

File: Source Code

studentServer.js

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

// Middleware for parsing JSON and form submissions
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Persistent Navigation Header Component
function getLayout(title, content) {
  return `<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${title} - Student Server</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Arial, sans-serif; }
    body { background: #f8fafc; color: #1e293b; padding: 24px; }
    .container { max-width: 800px; margin: 0 auto; background: #fff; border-radius: 10px; padding: 28px; box-shadow: 0 4px 12px #000000; }
    header { border-bottom: 2px solid #3b82f6; padding-bottom: 16px; margin-bottom: 20px; }
    h1 { color: #0f172a; font-size: 24px; margin-bottom: 12px; }
    nav { display: flex; gap: 10px; }
    nav a { text-decoration: none; }
    nav button { padding: 8px 16px; background: #3b82f6; color: #fff; border: none; border-radius: 6px; font-weight: 600; cursor: pointer; }
    nav button:hover { background: #1d4ed8; }
    nav button.secondary { background: #e2e8f0; color: #334155; }
    nav button.secondary:hover { background: #cbd5e1; }
    .content { padding: 10px 0; }
    table { width: 100%; border-collapse: collapse; margin-top: 14px; font-size: 13.5px; }
    th, td { padding: 10px 14px; border: 1px solid #cbd5e1; text-align: left; }
    th { background: #f1f5f9; color: #0f172a; font-weight: 600; }
    tr:nth-child(even) { background: #f8fafc; }
    .badge { display: inline-block; padding: 3px 8px; border-radius: 4px; font-size: 11.5px; font-weight: 600; background: #e0f2f1; }
    footer { margin-top: 24px; padding-top: 12px; border-top: 1px solid #e2e8f0; font-size: 12px; color: #64748b; text-align: center; }
  </style>
</head>
<body>
  <div class="container">
    <header>
      <h1>Student Information System</h1>
      <nav>
        <a href="/"><button>Home</button></a>
        <a href="/students"><button>Students Directory</button></a>
        <a href="/about"><button class="secondary">About Application</button></a>
      </nav>
    </header>
    <main class="content">
      ${content}
    </main>
    <footer>
      Full Stack Web Development &bull; Course Code: 23CM4121 &bull; ANITS CSM
    </footer>
  </div>
</body>
</html>`;
}

// Route 1: GET / (Home Route)
app.get("/", (req, res) => {
  const content = `
    <h2>Welcome to Student Server</h2>
  `;
  getLayout("Student Information System", content);
});
```

Source Code (Continued)

```
<p style="margin: 12px 0; line-height: 1.6; color: #475569;">
  This application is a RESTful web service built with <strong>Express.js</strong> on top of Node.js.
  Use the navigation menu above to inspect student directory rosters, access application metadata, or query routes.
</p>
<div style="background: #f0fdf4; border-left: 4px solid #10b981; padding: 14px; border-radius: 4px; margin-top: 16px;">
  <h3 style="color: #065f46; font-size: 15px; margin-bottom: 4px;">Server Status: Active</h3>
  <p style="color: #047857; font-size: 13px;">HTTP Server listening on port ${PORT} &bull; Environment: Node.js v20+</p>
</div>
`;
res.send(getLayout("Home", content));
});

// Route 2: GET /students (Students Directory Route - At least 5 students)
app.get("/students", (req, res) => {
  const students = [
    { roll: "A24126552268", name: "V S Sravan M", branch: "CSM-B", cgpa: "9.50", email: "sravan@example.com" },
    { roll: "A24126552260", name: "Ravi Teja", branch: "CSM-B", cgpa: "9.20", email: "ravi@example.com" },
    { roll: "A24126552261", name: "Rahul Verma", branch: "CSM-B", cgpa: "8.90", email: "rahul@example.com" },
    { roll: "A24126552262", name: "Ananya Sharma", branch: "CSM-B", cgpa: "9.45", email: "ananya@example.com" },
    { roll: "A24126552263", name: "Priya Patel", branch: "CSM-B", cgpa: "9.10", email: "priya@example.com" },
  ];

  let rows = students
    .map(
      (s) => `<tr>
        <td><strong>${s.roll}</strong></td>
        <td>${s.name}</td>
        <td><span class="badge">${s.branch}</span></td>
        <td>${s.cgpa}</td>
        <td>${s.email}</td>
      </tr>`
    )
    .join("");

  const content = `
    <h2>Registered Students Directory (${students.length} Records)</h2>
    <p style="margin: 8px 0 16px 0; color: #64748b; font-size: 13px;">List of enrolled students retrieved via GET /students:</p>
    <table>
      <thead>
        <tr>
          <th>Roll No</th>
          <th>Student Name</th>
          <th>Branch & Section</th>
          <th>CGPA</th>
          <th>Contact Email</th>
        </tr>
      </thead>
      <tbody>
        ${rows}
      </tbody>
    </table>
  `;
  res.send(getLayout("Students", content));
});

// Route 3: GET /about (About Application Route)
app.get("/about", (req, res) => {
  const content = `
    <h2>About Student Server Application</h2>
    <p style="margin: 12px 0; line-height: 1.6; color: #475569;">
      A modular server-side HTTP web application developed using Express.js to demonstrate HTTP request dispatching,
      semantic REST endpoint routing, and dynamic HTML template composition.
    </p>
  `;
  res.send(getLayout("About", content));
});
```

7. CODE EXPLANATION

Method	Description
<code>app.get()</code>	Routes HTTP GET requests.
<code>res.send()</code>	Sends the HTTP response.

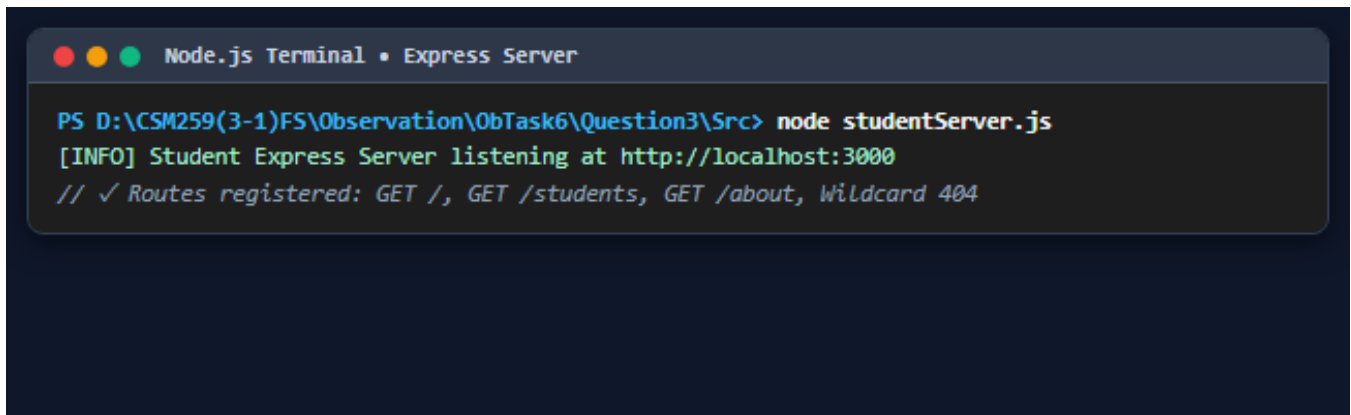
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Home Route	Navigate to /	Displays Home page	[Execution Verified]	Pass
TC-02	Students Route	Navigate to /students	Displays Students directory	[Execution Verified]	Pass

9. OUTPUT

Execution Output

A screenshot of a Node.js Terminal window titled "Node.js Terminal • Express Server". The terminal shows the command "node studentServer.js" being executed from the directory "D:\CSM259(3-1)FS\Observation\0bTask6\Question3\Src". The output consists of two lines: "[INFO] Student Express Server listening at http://localhost:3000" and "// ✓ Routes registered: GET /, GET /students, GET /about, Wildcard 404".

```
Node.js Terminal • Express Server

PS D:\CSM259(3-1)FS\Observation\0bTask6\Question3\Src> node studentServer.js
[INFO] Student Express Server listening at http://localhost:3000
// ✓ Routes registered: GET /, GET /students, GET /about, Wildcard 404
```

Execution Output

10. RESULT

The Express.js application was successfully developed and tested.

Submission Package Structure:

Observation task 6/Question3/